

Vision-Based Drone Chase

Deep reinforcement learning on a DJI Tello:
the algorithms, the three-tier simulation method,
and the ROS 2 flight stack

A drone that chases another drone using nothing but a camera. The policy that decides where to point is **learned**; the rule that decides how fast to close is **hand-written**; and everything is proven in three simulators of increasing realism before a propeller turns. This document explains every piece of that, in order, with the numbers each stage actually produced.

Reinforcement-learning verdict (20 runs, 4 algorithms × 5 seeds)	ACCEPT
Tier B physics agreement (A↔B replay gate, real Gazebo)	8.0 px vs 40 px tolerance
Tier C follow mission, vision in the loop	100 % over 1,500 frames
Detector precision (with drone-free frames in the exam)	0.996
Verified interception in the rendered warehouse	3.6 m chase, caught at 0.48 m
Flight-stack nodes implemented and tested	6 nodes, 18 tests
Total automated tests passing without hardware	105

What this document is careful about

Every number here came from a run on this hardware. Where a value is a design choice rather than a measurement it is marked **[D]**; where it is a placeholder that real flight must replace it is marked **[!]**. Section 9 states plainly what the system has *not* yet proven.

Contents

1	The problem, and the shape of the answer	what it does, and who decides what
2	Why three simulators	the method, and what each tier can and cannot prove
3	The one contract	observation, action, forward law, checkpoint
4	Tier A — learning the behaviour	the MDP, the reward, TD3 vs DDPG, the protocol
5	Tier B — does the maths match physics?	Gazebo, the sign test, the $A \leftrightarrow B$ gate
6	Tier C — does it survive real vision?	warehouse, dataset, detector, closed loop
7	The flight stack	six ROS 2 nodes, in detail
8	Safety	what restrains the aircraft, and what cannot
9	Results, and what is not yet proven	the honest ledger
10	Bring-up procedure	the flight ladder, step by step
A	Reference tables	files, topics, parameters, commands

1 The problem, and the shape of the answer

One drone follows another using a single camera and nothing else.

The task is deceptively simple to state: a DJI Tello watches the world through its forward camera, finds another drone in the image, and flies so as to keep it centred at a chosen distance — or, in the intercept variant, closes until it reaches it. There is no GPS, no motion capture, no depth sensor and no second camera. The only signal is a box around a small grey object in a 960×720 frame that arrives between 150 and 350 ms late.

1.1 Two learned things, and one thing that is not learned

It is worth being precise about this from the first page, because it is the most common misunderstanding of the system. There are **two trained models** and they do entirely different jobs:

- **The eyes** — a YOLO object detector. Input: a camera frame. Output: a box. It is trained by ordinary supervised learning on labelled images (Section 6).
- **The brain** — a reinforcement-learning policy. Input: where the box is and how it has been moving. Output: how to move the aircraft. It is trained by trial and error against a simulated target (Section 4).

And there is a third component that is emphatically *not* learned: the **forward-speed law**. The policy controls only two axes — vertical speed and yaw rate, the two axes that keep the target in frame. How fast the aircraft moves toward or away from the target is computed by a short, auditable, hand-written rule.

Why the forward axis is deliberately not learned

Monocular range is weak: the only distance signal is how wide the box is, and that estimate degrades with the square of distance. The forward axis is also the one that decides collision energy. A monotonic, clamped rule on that axis can be read, reasoned about and bounded by a human in a way a network cannot.

Meanwhile the genuinely hard perceptual-control problem — keeping a small, fast, intermittently-detected object inside the frame despite a third of a second of delay — is exactly what reinforcement learning is good at, and exactly what the reward measures.

This split also survives deployment unchanged: in the flight stack of Section 7 the learned part is `chase_policy` and the hand-written part is `chase_depth_rule`, and they own disjoint axes of the command by construction.

1.2 The aircraft is part of the problem

A stock Tello is a hostile platform for closed-loop vision, and every design decision downstream is shaped by its limits. These are measured properties of this airframe, not estimates:

Property	Value	Consequence for the design
Video latency	150–350 ms	The policy must be delay-aware; it sees the past, not the present
Video stamp	applied at publish, not capture	True latency is under-reported by the decode+poll time
Command interface	normalised sticks in $[-1, 1]$, scaled to ± 100	The policy output range and the aircraft input range coincide exactly
Command dead-man	0.35 s	Commands must be published continuously at ≥ 10 Hz, including zeros
Gyroscope	none	No angular-rate feedback; attitude only, in whole degrees at 10 Hz
Position	none (no GPS, no mocap)	A lateral geofence is impossible — see Section 8
Target size	0.098 m body / 0.180 m prop span	At 4 m the target is ~23 px wide: a small-object detection problem

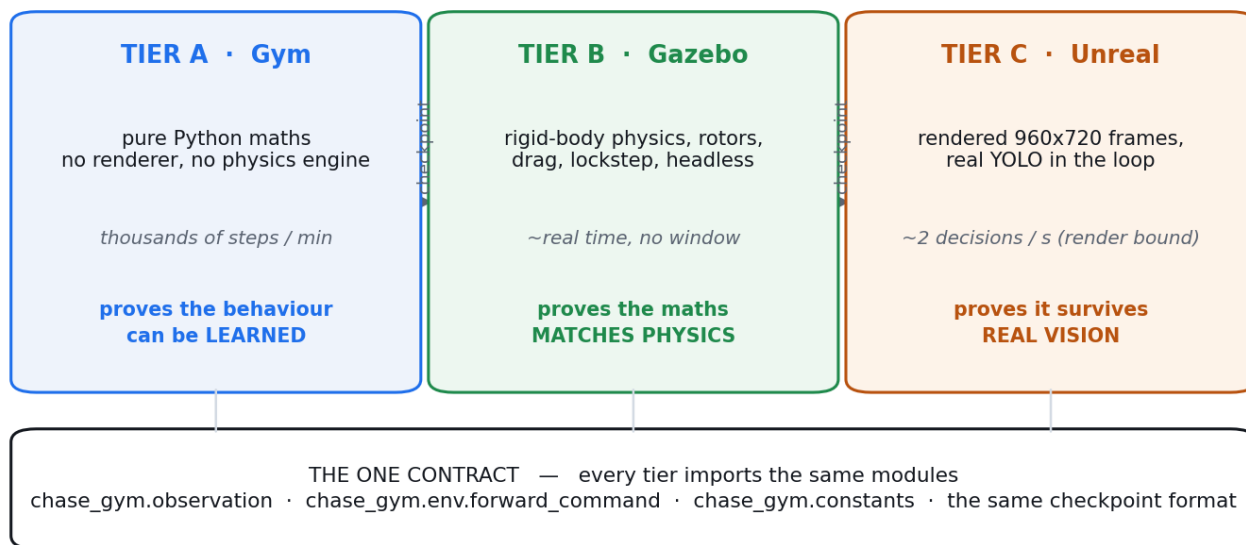
The dead-man is a feature, not an obstacle

Because the Tello latches its last stick command forever, the driver zeroes the sticks if no command has arrived for 0.35 s. That means a crashed or hung policy node *stops* the aircraft rather than leaving it flying. The flight stack is built around this: a zero command is a message that must be *sent*, never an absence of messages.

2 Why three simulators

Three different questions, three different fidelities, one shared contract.

Before the aircraft flies, three separate questions must be answered, and no single simulator answers them well. Learning needs millions of attempts, which rules out anything that renders. Checking the physics needs a rigid-body engine. Checking the vision needs photorealistic pixels. So the work is split into three tiers, each of which is fast at exactly the thing it must prove.



The three tiers. They never talk to each other at run time: they are connected by shared code and by files (a trained checkpoint, a dataset, a detector).

Tier	What it is	Speed	The question it answers
A	Pure-Python kinematics and a simulated detector. No renderer, no physics engine, no ROS.	thousands of steps/min	Can the behaviour be learned at all?
B	Gazebo Harmonic: real rotors, inertia, drag. Headless, stepped in lockstep. Measurements come from projecting ground truth, not from a camera.	~real time	Does the maths match real physics?
C	Unreal Engine 5.7 via Project AirSim: rendered 960×720 frames, the real YOLO detector in the loop.	~2 decisions/s	Does it survive a real camera and a real detector?

2.1 Why no tier has a window

Tier A and Tier B render nothing, deliberately. Tier A is arithmetic; there is nothing to draw. Tier B has no camera at all — its "detector" is an oracle that projects the true relative position into image coordinates through the same pinhole model the real camera obeys, then delays and corrupts it. Rendering would add cost and no information, because Tier B is not the tier that tests vision. Only Tier C draws pixels, because in Tier C the pixels *are* the experiment.

2.2 What each tier cannot prove

Being explicit about the ceiling of each tier is what makes the ladder meaningful:

- **Tier A cannot prove its own realism.** It can only show that a policy learns to solve the world it was given. Whether that world resembles a flying machine is precisely what Tier B tests.
- **Tier B cannot say anything about perception.** Its measurements are derived from truth. A detector that hallucinates, or misses small targets, is invisible here.
- **Tier C cannot prove the real world.** Its images come from a renderer, and a detector trained on them has only ever seen that renderer. This is the gap that remains open at the end of this document (Section 9).

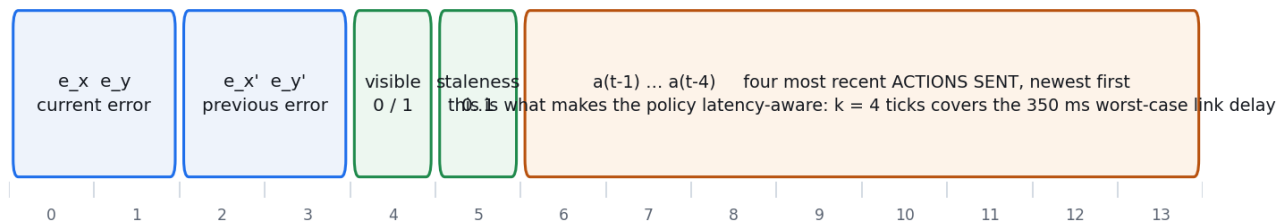
3 The one contract

What makes a checkpoint trained in Tier A run unchanged in Tier B, Tier C and on the aircraft.

The tiers are interchangeable only because they all speak the same four interfaces, and because those interfaces exist once in code and are *imported* rather than reimplemented. This is the single most important structural decision in the project: it makes train/deploy skew impossible rather than merely unlikely.

3.1 The observation — 14 numbers

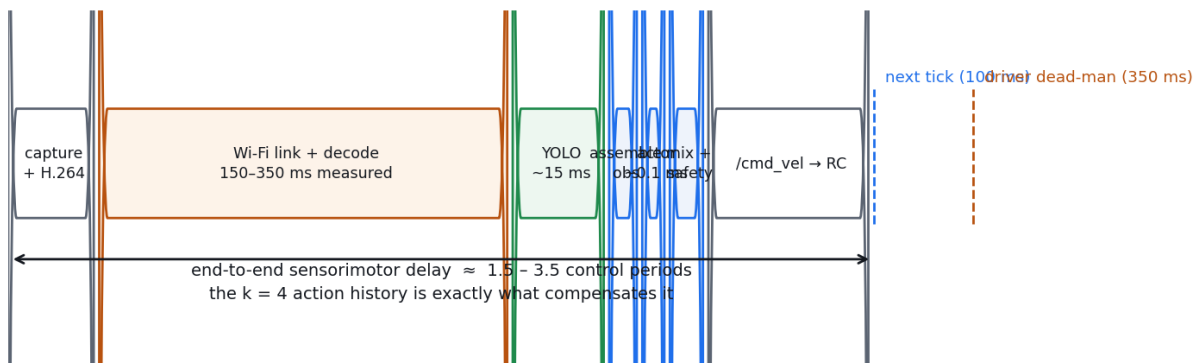
Every tier and the aircraft build the policy's input with the same function, `ObservationAssembler.assemble()`:



The observation vector. Errors are normalised to $[-1, 1]$ by half the frame, so the policy is resolution-independent.

Two details in that layout carry a great deal of weight:

- **On a miss, the errors are zeroed — never held.** It is tempting to repeat the last known position when the detector fails. That trains the policy to chase ghosts: it learns to act confidently on stale information, which is exactly the behaviour that flies a drone into a wall after the target leaves the frame. Instead the errors go to zero and `visible` goes to 0, so "I cannot see it" is a distinct, learnable state.
- **The last four actions are part of the input.** This is what makes the policy *latency-aware*. The image describes the world as it was 150–350 ms ago; the action history describes what the aircraft has done since. With a 10 Hz control rate, $k = 4$ ticks spans 400 ms — slightly more than the worst-case delay, by construction.



Where the delay lives. The policy never sees the present; the action history is how it compensates.

3.2 The action — two sticks

The actor emits two numbers in $[-1, 1]$. They map onto the aircraft with **no conversion layer whatsoever**, which is a rare piece of luck worth protecting:

```
action[0] = a_v    -> linear.z    (vertical stick, +up)
action[1] = a_yaw  -> angular.z   (yaw stick, +counter-clockwise)
```

sign convention, verified by unit test and re-tested per tier:

- +angular.z (CCW) moves the target box RIGHT in the image (+u)
- +linear.z (up) moves the target box DOWN in the image (+v)

Because `/cmd_vel` already takes normalised sticks in $[-1, 1]$ and the driver scales them to the SDK's ± 100 internally, the network's output range and the aircraft's input range are the same interval. Inserting any rescaling between them would silently change what the policy's learned magnitudes mean, so none exists, and the checkpoint loader refuses any policy whose declared action scale is not exactly 1.0.

Signs are tested, never assumed

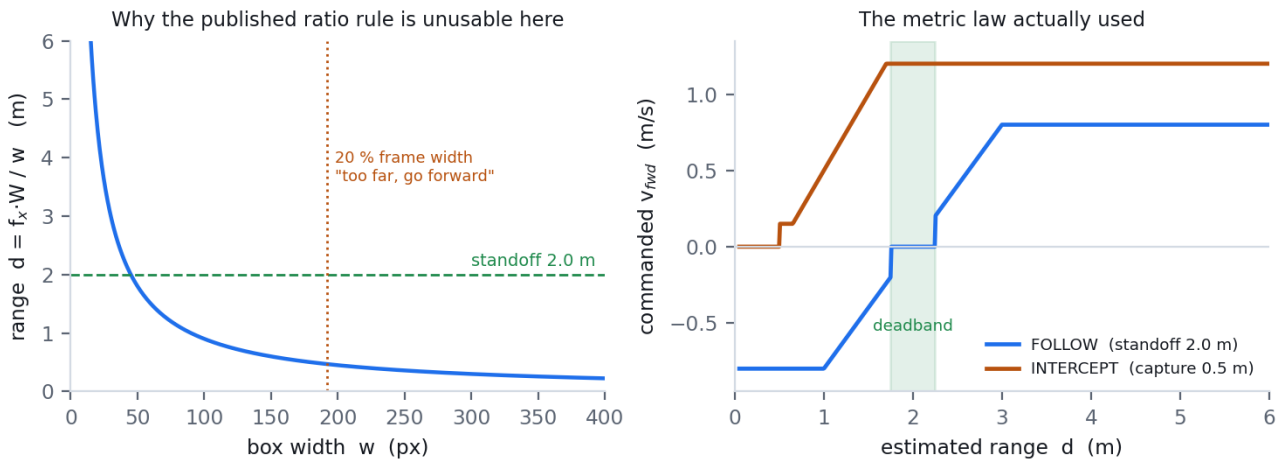
Three simulators plus one aircraft is four independent chances for a silently flipped axis, and a flipped yaw turns a chase into an escape. Each tier therefore runs a sign test that commands one axis and asserts the direction the target moves in the image. The flight stack ships two such tests as automated integration tests (Section 7.7).

3.3 The forward law — range from a box width

Because the target is a known object, the width of its box is a distance measurement through the pinhole relation:

$$d = f_x \cdot W / w \quad f_x = 919.42 \text{ px}, \quad W = 0.098 \text{ m (Tello body width)}$$

The source work commands forward motion from the *fraction of the frame* the box covers, with thresholds at 20 % and 55 %. Against a Tello-sized target and this camera those thresholds are catastrophic: 20 % of the frame width is not reached until roughly 0.47 m, and 55 % corresponds to about 0.17 m. A faithful port would therefore command "too far — move forward" at *every safe separation* and stop only when the aircraft were a few tens of centimetres apart.



Left: the published ratio thresholds mapped onto real range for this target — a collision policy. Right: the metric law actually used.

The law that is used instead, identical in all four places it runs:

FOLLOW	err = d - standoff err < 0.25 m otherwise	(standoff = 2.0 m) -> v = 0 (deadband) -> v = clip(0.8 * err, -0.8, +0.8) m/s
INTERCEPT	d <= r_cap (0.5 m) otherwise	-> v = 0 (inside: terminal) -> v = clip(1.0 * (d - r_cap), 0.15, 1.2) m/s
NO DETECTION		-> v = 0 (hold; never coast)

3.4 The checkpoint — auditable or it is folklore

A saved policy is not just weights. Each bundle carries the exact observation specification and its hash, the action map, the control rate, the resolved environment config, the git commit, the seed, the environment-step count and the library versions. Any consumer that cannot reproduce the observation hash **refuses to load it**. That refusal is the entire sim-to-real audit trail in one mechanism: an actor is meaningless without the exact input contract it trained under, and a silent mismatch would fly a network on numbers that mean something different from what it learned.

```
obs_spec_hash   = '3e902aea0712a3c9'
action_map      = {'order': ['linear.z', 'angular.z'],
                   'scale': 1.0, 'stick_range': [-1.0, 1.0],
                   'signs': '+angular.z (CCW) moves box +u; +linear.z (up) moves box +v'}
control_rate_hz = 10.0
git_sha         = 'a4c71c7111b76eb662cf9eddc26bb3ff94e6493b'
```

4 Tier A — learning the behaviour

The Markov decision process, the reward, and the algorithm that solves it.

Tier A is where the policy is actually trained. It is a pure Python environment: no renderer, no physics engine, no ROS. That austerity is the point — it runs thousands of steps per minute, which is what makes twenty independent training runs affordable.

4.1 The decision problem

Element	Definition
State (observed)	The 14-dim vector of Section 3.1 — delayed, noisy, sometimes absent. The policy never sees true geometry.
Action	$a \in [-1, 1]^2 \rightarrow$ vertical stick, yaw stick. Scaled by $v_max = 1.5$ m/s and $\omega_max = 1.5$ rad/s.
Step	$\Delta t = 0.1$ s — the deployment control period, not a tuning knob.
Episode	300 steps (30 s), or until the target leaves frame, or capture.
Discount	$\gamma = 0.99$

The simulator is deliberately unkind

Measurements are delayed by a per-episode base drawn from $U(150, 350)$ ms plus per-frame jitter; boxes are jittered by ~ 2 px; detections drop out with a probability that rises from 2 % for a large box to 30 % for a small one; and a two-state Markov chain produces multi-frame *burst* dropouts averaging half a second. A policy that only works on clean data never survives training here — which is the entire purpose.

4.2 The reward

The reward is computed from **truth**, not from the detection. This is a deliberate asymmetry: the agent must act on a corrupted view but is graded on what actually happened, so it cannot earn reward by fooling its own sensor.

```

d_px = pixel distance from the frame centre to the TRUE box centre

tracking    r_track = (100 - d_px)/100          if d_px <= 100
              = -0.25 * (d_px - 100)/100      otherwise

smoothness - 0.05 * || a_t - a_{t-1} ||^2      (anti-judder)
loss        - 5.0                             on the step the target exits frame

INTERCEPT additionally:
  shaping    + gamma * Phi(s_{t+1}) - Phi(s_t),  Phi(s) = -0.1 * range_m
  capture    + 10.0                             on reaching 0.5 m in frame

```

Two of those terms deserve comment. The **tracking** term is continuous and crosses zero at exactly 100 px: inside that disc the agent is paid, outside it is charged, and the slope outside is gentler than inside so that a briefly-off-centre target does not dominate the return. The published version of this reward has a 25-point *cliff* at the threshold — the reward jumps discontinuously from 0 to -25 as the target crosses 100 px. That discontinuity is reproduced exactly in the faithful variant, for comparison, and repaired in the one that flies.

The **shaping** term uses the potential-based form $F = \gamma\Phi(s') - \Phi(s)$, which is provably unable to change the optimal policy — it only makes the gradient easier to find. The capture bonus, by contrast, *does* change the optimum, deliberately: capture is the task.

4.3 One environment step, in order

The order of operations inside a step is itself a contract, because it determines what the policy can possibly know:

1. the target moves (it acts first; we react)
2. forward speed from the DELAYED measurement of the PREVIOUS step
-- the deployed depth rule also acts on detections, never on truth
3. the follower integrates a first-order velocity lag ($T_{\text{lag}} \sim 0.25$ s)
4. TRUE geometry is projected through our calibration $\rightarrow u, v, w_{\text{px}}$
5. if the target is in frame, truth enters the delay line
6. a measurement is SAMPLED from the delay line, Delta seconds old
7. that measurement is corrupted (jitter / dropout / burst)
8. `record_action(a)` THEN `assemble(meas)` -- the ordering contract
9. terminals are decided from TRUTH; the step budget is truncation

Truncation is not termination

When an episode ends merely because the 300-step budget ran out, the value bootstrap must continue through the cut — the world did not end, the clock did. Conflating the two teaches the agent that time itself is fatal and systematically depresses the value of every long-horizon action. The two flags are mutually exclusive by construction here.

4.4 The algorithm: TD3, and why not plain DDPG

The source work uses DDPG. DDPG is an actor-critic method for continuous actions: a *critic* learns the value of a state–action pair, and an *actor* is trained to output the action the critic likes most. Its well-known failure is that the critic systematically **overestimates**: the actor exploits whatever the critic happens to over-value, the critic then trains on those inflated targets, and the pair can spiral. TD3 is DDPG plus three specific repairs.

Twin critics Q_1, Q_2	Delayed policy updates	Target policy smoothing
target uses $\min(Q_1, Q_2)$ $\rightarrow$ kills the systematic overestimation of DDPG	actor + targets updated every $d = 2$ critic steps $\rightarrow$ critic leads, actor follows	$a' = \text{clip}(\mu'(s') + \epsilon)$ $\epsilon \sim \text{clip}(N(0, \sigma), -c, c)$ $\rightarrow$ no sharp-peak exploits

The three differences between TD3 and DDPG.

As implemented, the update is exactly:

```
target action  a' = clip( mu'(s') + clip(eps, -c*s, +c*s), -s, +s )
                eps ~ N(0, (sigma_t * s)^2),  sigma_t = 0.2, c = 0.5

label          y  = r + gamma * (1 - d) * min( Q'_1(s', a'), Q'_2(s', a') )

critic loss    L_c = SUM_i  MSE( Q_i(s, a), y )          (sum, not mean)
actor  loss    L_a = - mean( Q_1(s, mu(s)) )            (Q1 only, never the min)

every d = 2 gradient steps:  actor step, then
                             theta' <- (1 - tau) theta' + tau theta,  tau = 0.005
```

Three details in that listing are easy to get wrong and are worth naming. The smoothing noise exists **only inside the label** — it is never added to the action the aircraft takes. The actor is trained against **Q^π alone**, not the minimum, because the minimum is a pessimistic estimate appropriate for labelling, not for ascent. And the soft update is counted in *gradient* steps and applied only on delayed steps, so the targets move at half the critic's rate.

Three algorithm variants were trained head to head, so that the choice is a measurement rather than a preference:

Arm	What it is	Why it is in the comparison
td3	All three repairs active.	The candidate for deployment.
ddpg_repaired	DDPG in the same environment, same observation, same reward — without the TD3 repairs.	Isolates the algorithm from the environment: any difference is attributable to the repairs alone.
ddpg_faithful	An exact reproduction of the published method, including its raw-pixel observation, its ± 60 action scale, its reward cliff and its treatment of timeouts as terminal.	Establishes what the original actually does, so improvements are measured against it rather than asserted.
sb3_td3	An independent third-party TD3 implementation.	Guards against a bug in our own trainer flattering our own result.

Critically, **the arms are configuration, not separate code**. There is one trainer class; DDPG is TD3 with the twin critic, the smoothing and the delay switched off. Only five places in the whole package branch on the arm name. That is what makes the comparison honest: any difference between arms cannot be an accident of two divergent implementations.

Setting	td3	ddpg_repaired	ddpg_faithful
Critics	2	1	1
Target smoothing σ^2 / clip	0.2 / 0.5	off	off
Policy delay	2	1	1
τ (soft update)	0.005	0.005	0.001
Learning rate (actor/critic)	3e-4	3e-4	1e-3, clipnorm 1
Hidden layers	[256, 256]	[256, 256]	[16,16,16] / [32,32,32]
Observation	14-dim	14-dim	2-dim raw pixels
Action scale	1.0	1.0	60.0
Exploration noise	Gaussian 0.3→0.05	same	0U, post-scale, unclipped
Timeout treated as terminal	no	no	yes
Curriculum / prefill	yes / 20 eps	yes / 20 eps	no / none

A quantified explanation of why the faithful arm struggles

Its Ornstein–Uhlenbeck exploration noise has a stationary standard deviation of about 0.55, and it is added *after* the $\times 60$ action scaling and never clipped. So its exploration perturbs an action range of ± 60 by well under one per cent. It is not a bug in this reproduction — it is faithful to the original, and it is a measurable reason the method behaves as it does.

The faithful arm is quarantined, on purpose

Because it emits a 2-dimensional raw-pixel observation and an action map of (image_x, image_y) scaled by 60, it is *structurally* incompatible with the flight stack. The flight loader refuses it by name. It exists to be measured, never to be flown.

4.5 Training protocol

Curriculum. Training begins against a stationary target and widens in four stages. The gate is a *rolling* one that can also step back down, and — importantly — it is measured on a frozen full-difficulty evaluation suite, not on the restricted training distribution, so "progress" cannot be an artefact of an easier exam.

Stage	Target motion families	Speed cap	Advance / demote
0	static	0.0 m/s	≥ 0.90 / ≤ 0.70
1	static, constant velocity	0.4 m/s	≥ 0.90 / ≤ 0.70
2	constant velocity, vertical + horizontal oscillation	0.6 m/s	≥ 0.90 / ≤ 0.70
3	all five, including aggressive	1.0 m/s	—

- **Warm start.** The replay buffer is pre-filled with 20 episodes flown by the P-controller baseline — a few thousand transitions — so the agent begins from "someone who can already roughly chase" rather than from noise. The demonstration seeds are drawn from a block disjoint from both training and evaluation.
- **Exploration.** Gaussian noise decaying linearly from $\sigma = 0.3$ to 0.05, added *before* the clip to $[-1, 1]$, on top of a 1,000-step uniform-random warm-up.
- **Selection.** The saved checkpoint is the one with the best evaluation time-in-view *on the moving families* — not the most recent, and not one flattered by static-target scores. Return breaks ties within a 0.005 band.
- **Stopping.** Training halts on an evaluation plateau: five consecutive evaluations without improvement, and only after the curriculum has reached its final stage. In practice runs stopped between 60,000 and 95,000 steps against a 200,000-step budget.
- **Reporting.** The headline numbers come from a **held-out seed block** 500,000 away from the selection suite, scored on the selected checkpoint rather than the final weights — because selection, the curriculum and the plateau rule all consumed the selection suite.

4.6 The acceptance rule

A result is accepted only if the **lower** bound of the arm's confidence interval exceeds the **upper** bound of the P-controller baseline's interval, on the *moving* target families. Comparing means would let noise decide; comparing on static targets would flatter everything. Five seeds per arm make the interval meaningful.

Result: ACCEPT

All reinforcement-learning arms beat the classical P-controller on moving targets under the interval rule. The deployable follow policy scored a held-out time-in-view of 99.7 %; the intercept policy 98.1 %.

5 Tier B — does the maths match physics?

Gazebo Harmonic, headless and in lockstep.

Tier A approximates flight with a first-order velocity lag. Gazebo simulates rotors, inertia and drag. If a policy only works in the approximation it is worthless, so Tier B is the referee between the two worlds. It runs the same environment interface, the same observation module and the same forward law — only the body dynamics are replaced.

5.1 The three checks

Check	What it does	Result
Sign test	Commands one axis at a time and asserts the direction the target moves in the image, through the entire chain.	PASS
Lag calibration	Measures the simulated aircraft's velocity step response.	0.54–0.84 s measured
A↔B replay gate	Replays identical command sequences in Tier A and in Gazebo and compares the resulting image-space trajectories pixel by pixel.	8.0 px median vs 40 px tolerance

The replay gate is the certificate that makes the whole ladder meaningful: it says the world the policy trained in agrees with rigid-body physics to within a few pixels of image-space truth.

Two defects this gate found

The gate could not run at all until a reset bug was fixed: after teleporting both aircraft into place, the world was given only **12 milliseconds** to settle before the first comparison, so the very first sample was taken mid-transient. It now holds for 1.5 s while re-arming the controllers.

Fixing that exposed a second, subtler defect. The delay queue compared timestamps with a bare $\leq$. When the latency is an exact multiple of the control period, the comparison lands precisely on a stamp and its outcome depends on accumulated floating-point error — error that *grows* with simulated time. Starting the clock later silently flipped the comparison from inclusive to exclusive and froze the observation at its pre-rolled value. A nanosecond of slack makes the knife edge deterministic.

Tier B has no window and no camera, by design. Its measurements come from projecting ground truth through the same pinhole model the real camera obeys, then delaying and corrupting it exactly as Tier A does. It also runs faster than real time in lockstep, so a rendered view would be an unwatchable stutter as well as an irrelevance.

6 Tier C — does it survive real vision?

Unreal Engine, a rendered warehouse, and a detector trained on it.

Tier C is the only tier with pixels, because in Tier C the pixels are the experiment. It answers the question the other two cannot: does the policy still work when its input comes from an actual object detector looking at an actual image?

6.1 A warehouse, built at run time

The deployment target is indoor flight, so the scene is an enclosed warehouse — floor, ceiling, four walls, shelving racks down both sides of a central aisle, loaded with crates, boxes and barrels, lit by ceiling lights whose intensity is genuinely driven by the requested lighting class. This matters more than it sounds: a detector inherits the domain it is trained on, and a detector trained on an outdoor test level will not find a drone in a warehouse.

6.2 A dataset with perfect labels

The target is placed on a grid of ranges (0.5–6 m), azimuths, elevations and headings, in three lighting classes. Each frame is saved with a label taken from the engine's own ground truth — so the labels are exact by construction, with no human error and no annotation cost. That produced **3,003 labelled frames**.

6.3 The phantom lesson

The first detector (v1) scored a precision of 0.995 and looked excellent. It was not. Its validation set contained only images *with a drone in them*, so "precision" had never been tested against the one thing that matters in flight: an empty room. The moment the chase flew off the aisle centre line, v1 produced large, high-confidence "drone" boxes on dark walls and shelf shadows — and because range is inferred from box width, a huge phantom box reads as a target a few centimetres away. Every hallucination became a fake capture.

A detector that has never seen a drone-free image cannot say "no drone"

The fix was to add **360 verified target-free frames** (~11 % of the set) captured by a patrol that hides the target under the floor and photographs walls, racks and dark corners. Retrained, v2 holds precision 0.996 *with those frames inside the exam*, and all three frames that fooled v1 come back clean.

Detector	Precision	Recall	mAP50	Backgrounds in validation?
v1	0.995	0.711	0.732	none
v2	0.996	0.705	0.724	72 frames

Recall of ~0.70 is not a defect to be tuned away: the misses are targets below roughly 30 px wide, i.e. beyond about 4 m at extreme angles. That is a genuine physical limit of a 0.098 m object in a 960 px frame, and the chase policy's job is to keep the target larger and more central than that.

6.4 Vision in the loop

The final Tier C test closes the loop with nothing mocked: rendered frame → the trained YOLO → the same observation assembler → the trained actor → stick commands → the engine flies → next frame.

Mission	Result	Reading
Follow (the deployment task)	100 % detection, 5 × 300 steps	The policy held the target inside the detectable envelope for 1,500 consecutive frames. This is the mission the aircraft will fly.

Mission	Result	Reading
Intercept	verified capture: 3.6 m chase, caught at 0.48 m	Genuine — the target's rotors are inside the detection box on the saved frame. Other episodes tracked perfectly but ran out the 300-step clock.

Why the intercept capture rate is a lower bound, not a score

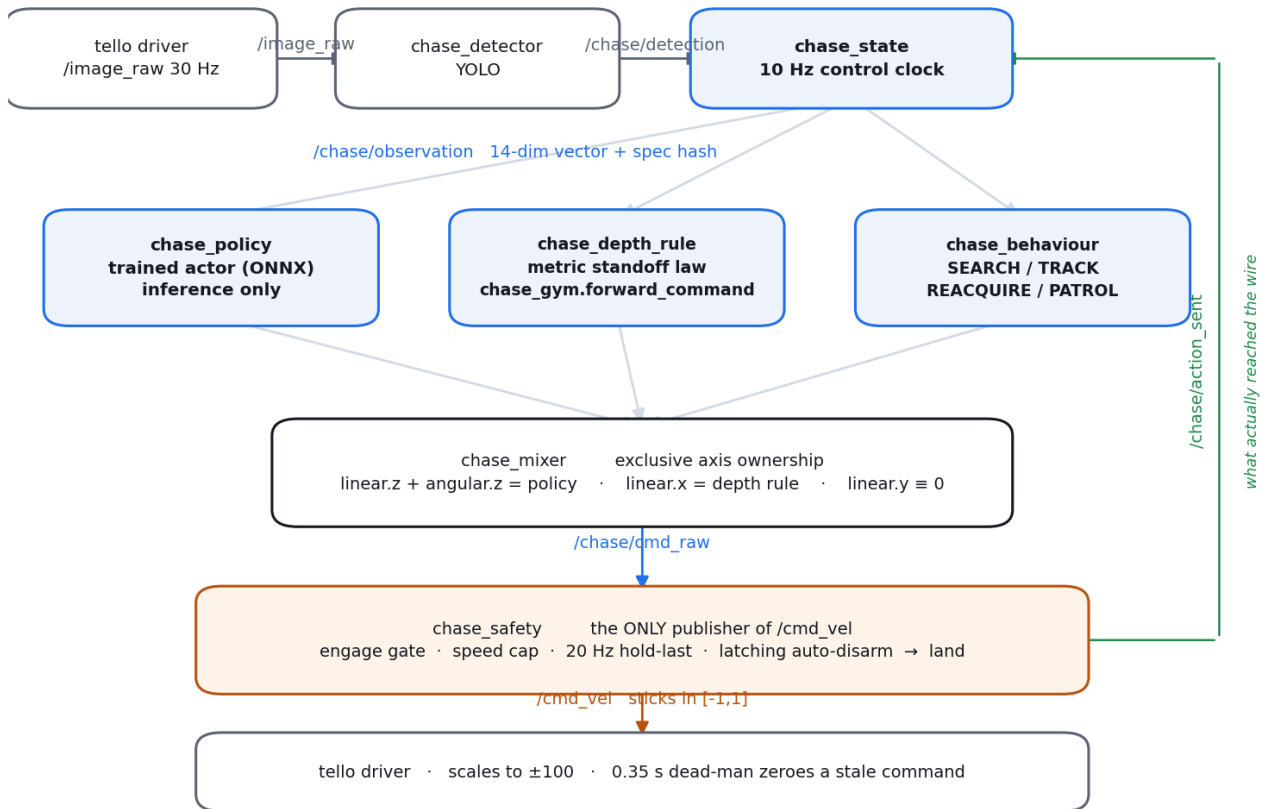
The evaluation loop is rendering-bound: each tick costs a render plus a YOLO inference, so the aircraft receives roughly two commands per second and spends most of the wall clock decelerating. The closing law also tapers to 0.15 m/s near the target. The result is that the last metre eats the step budget. The deployed stack streams commands continuously at 10 Hz and does not have this problem.

Getting an honest number here took three separate corrections, each of which had produced a *convincing* false positive: episodes that "captured" on their first frame (the previous episode's momentum carried the aircraft onto a freshly-placed target), captures on giant phantom boxes (Section 6.3), and an early 80 % capture rate that was all three artefacts stacked. The working rule that emerged is simple: **a number is not a result until the frame behind it has been looked at.**

7 The flight stack

Six ROS 2 nodes that put the trained policy on the aircraft.

The flight software is deliberately split into six small nodes with one responsibility each. The split is not decoration: it means the observation definition can change without touching perception, the policy can be swapped for a hand-written controller without touching anything else, and — most importantly — the component that restrains the aircraft runs in a *different process* from the components that command it.



The flight graph. Exactly one node publishes `/cmd_vel`.

7.1 chase_state — the observation clock

Owens the 10 Hz control tick. On each tick it records the action that was actually *sent*, assembles the 14-dim observation from the newest detection, and publishes it. Two details are load-bearing:

- **The tick is a timer, not a detection callback.** Detections stop arriving precisely when the target is lost — which is exactly when the staleness counter must keep climbing. A detection-driven clock freezes staleness at the moment it matters.
- **Detections are consumed once.** Deciding freshness by age alone lets a single detection be re-used on many ticks, which both double-counts it as "visible" and stalls staleness at zero. This was a real defect, caught by the integration tests in Section 7.7.

7.2 chase_policy — inference only

Loads the actor and runs one forward pass per tick. No noise, no learning, no exploration: the aircraft flies a frozen checkpoint, because an updating policy on a live aircraft has no safety argument. Inference uses the exported ONNX graph; the torch bundle is still read, because it is the only artifact carrying the observation hash and action map that authorise the graph to fly at all. A mismatch is a hard refusal, not a warning.

The same node can fly a hand-written P-controller instead (`baseline:=p_controller`), which is how the flight ladder begins — see Section 10.

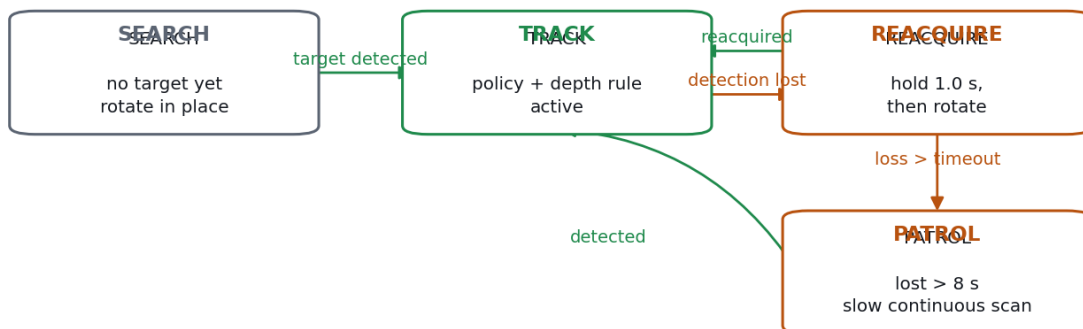
7.3 chase_depth_rule — the forward axis

Imports `chase_gym.env.forward_command` directly: the same function Tier A stepped and Tier B replayed. It publishes metres per second; converting to sticks is the mixer's job. It ignores the detector's own published range entirely, for a reason worth stating plainly.

A silent 1.84× range error, designed out

The detector node defaults its reference width to 0.180 m (the *propeller span*) while the trained policy's config uses 0.098 m (the *body width*). Both are correct measurements of the same aircraft, and mixing them scales every range estimate by 1.84. This stack therefore derives range from the box width with the checkpoint's own value, and the launch file sets the detector to match so that a human reading the topic is not misled either.

7.4 chase_behaviour — the mission state machine



Every timeout here is a design choice, not a measurement: the source work names the four knobs and fixes none of them.

SEARCH / TRACK / REACQUIRE / PATROL. REACQUIRE holds position before it rotates — it does not coast toward a target it can no longer see.

Every timeout in that diagram is a **[D] design choice**. The source material names exactly these four knobs — lost-frame count, loss timeout, search rate, patrol pattern — and fixes none of them. The values shipped are conservative starting points, and they are the first thing to tune in flight. One of them does have provenance: the 1.0 s loss timeout matches the staleness normaliser baked into the checkpoint's observation spec, i.e. the value the policy was actually trained against.

The search rate is chosen against the *detector*, not against the airframe: at a quarter stick the aircraft sweeps its 55° field of view in about 2.6 s, which gives the detector several frames on anything it crosses. Sweeping faster than the detector can see is the classic way to rotate straight past the thing you are looking for.

7.5 chase_mixer — exclusive axis ownership

Two controllers writing one axis is the classic route to an uncontrollable aircraft, so ownership is structural rather than a matter of discipline:

<code>linear.z</code>	(up/down)	<code>chase_policy</code>	learned
<code>angular.z</code>	(yaw)	<code>chase_policy</code>	learned [see precedence]
<code>linear.x</code>	(forward/back)	<code>chase_depth_rule</code>	hand-coded
<code>linear.y</code>	(lateral)	UNUSED --	always exactly 0.0

Precedence. The design set never defined how the behaviour mode combines with the policy, so this stack defines it: outside TRACK the behaviour node owns yaw and the policy is ignored. The reasoning is that outside TRACK there is no target in frame, the observation errors are therefore zeroed, and the policy's output carries no information about where to point.

Units. The policy axes pass through *unscaled*: the checkpoint declares a stick range of $[-1, 1]$, `/cmd_vel` consumes exactly that, and the driver scales to the SDK's ± 100 itself. Only the forward axis is converted, from m/s to sticks — and that conversion needs a number this project does not yet have.

[!] The one unmeasured number in the control path

Converting the depth rule's metres per second into a stick requires the airframe's full-stick forward speed. It has never been measured on this aircraft. The simulator assumed 1.5 m/s, but setting this *below* the truth makes the aircraft fly faster than intended, and the manufacturer quotes 8 m/s — so the shipped default is a deliberately **high** 6.0 m/s, and the supervisor caps the forward axis separately at 0.15 of full stick. Measuring the real value is item 1 of the bring-up procedure.

7.6 chase_safety — the supervisor

The only publisher of `/cmd_vel`, running in its own process so that a controller bug cannot disable the thing that restrains it. Covered in full in Section 8.

7.7 How it is tested without an aircraft

Seven integration tests start all six real nodes in one process (eleven more, in a separate file, attack the supervisor's fault logic — Section 8.2), inject synthetic detections and assert on what reaches `/cmd_vel`:

- A good detection while **disarmed** produces exactly zero on every axis.
- The lateral axis is **always** zero, structurally.
- No axis ever exceeds the speed cap, even at detector saturation.
- **Sign test, yaw**: a target right of centre yaws right.
- **Sign test, vertical**: a target below centre descends.
- A lost target holds the forward axis at zero — no coasting.
- A far target commands forward; a close one backs off.

These tests earned their keep immediately

The target-loss test failed on first run and exposed the consume-once defect described in Section 7.1 — a bug that would have made the aircraft believe it could still see a target it had lost, for as long as the timeout allowed.

7.8 Is Python fast enough? (measured)

No code runs on the Tello itself — it has no user-programmable computer. All six nodes run on the ground laptop, which sends stick values to the aircraft over Wi-Fi. The question is therefore whether Python on the laptop keeps up with a 10 Hz control loop. It was measured rather than assumed.

Step (per control tick)	Median	p99
Policy inference, ONNX	11.5 μ s	27 μ s
Observation assembly	10.8 μ s	32 μ s
Forward law	2.2 μ s	4 μ s

Step (per control tick)	Median	p99
Detection → measurement	1.1 μ s	2 μ s
Total compute	~26 μs = 0.03 % of the 100 ms period	

A C++ rewrite would save perhaps twenty microseconds per tick against a Wi-Fi video link that is 150–350 *milliseconds* late. The expensive work — YOLO inference and H.264 decoding — already runs in compiled CUDA/C++ kernels underneath Python, and the Tello driver this stack sits on is itself Python.

What *did* matter was the architecture. Measuring the six nodes as separate processes, from a detection being published to the command reaching `/cmd_vel`, exposed a defect that no language change would have fixed:

Design	Median latency	Worst case
Mixer and supervisor on their own timers	94–151 ms, <i>varied by launch</i>	125–200 ms
Mixer and supervisor react to each new input (current)	51–61 ms, consistent	~103 ms
... with all 16 CPU cores saturated	61 ms	103 ms

Timers were adding delay the policy never trained with

When the mixer and the supervisor each waited for their own clock tick, a fresh action sat idle for up to 100 ms and then up to 50 ms more — and because each timer started at a random moment, the total depended on when the stack was launched. Two clean runs of identical code measured 94 ms and 151 ms.

Both nodes now react the instant an input arrives, keeping their timers only as watchdogs. What remains is the state node's own 100 ms tick, which *is* modelled in training: the simulator also samples measurements at step boundaries. The longest gap between commands stayed at ~57 ms even under full CPU load — six times inside the driver's 350 ms dead-man.

Where C++ *would* become justified: moving the stack onto an onboard computer with little CPU (a Jetson or a microcontroller board), running the detector at high frame rates on CPU, or needing hard real-time guarantees that no Python executor can give. None of those apply to a laptop driving a Tello over Wi-Fi.

8 Safety

What actually restrains this aircraft — and what cannot.

The supervisor is the last node before the driver and the only publisher of the command topic. It is a separate process from the mixer on purpose: isolation means a bug in a controller cannot switch off the thing that limits it.

Mechanism	Behaviour	Default
Engage gate	Until an operator arms the stack the command on the wire is exactly zero. The policy runs, publishes and is recorded from the first second — it simply does not reach the aircraft.	starts DISARMED
Speed cap	Every axis clamped before it is sent, and a non-finite value becomes zero, never a deflection. On this airframe this is the safety envelope. The forward axis has its own, tighter cap because its stick scale is unmeasured.	0.6 stick; forward 0.15 [D]
Dead-man feed	Forwards each new command immediately, and republishes the last one at 20 Hz, so the driver's 0.35 s dead-man is always fed and a stalled upstream node stops the aircraft.	20 Hz + on arrival [V]
Auto-disarm	Latches disarmed and commands a land on any of: link loss, low battery, prolonged target loss, a target never acquired, altitude breach, a stalled state node, or /emergency. Every rule fails closed : a missing input is a fault.	see 8.1
Arming	Edge-triggered, and refused until telemetry exists and any latched fault has actually cleared.	operator act

The honest limitation: there is no geofence

This method estimates no absolute position. The aircraft knows its *height* — from the time-of-flight sensor and the SDK's own estimate — but nothing whatsoever about where it is horizontally. A lateral geofence is therefore not implementable, and this stack does not pretend to offer one.

What actually keeps the aircraft in the room is the speed cap, a short flight volume chosen by the operator, and a human holding the abort. Those should be described that way rather than overclaimed. The altitude limits *are* real, because height is measured. There is also no hardware kill line on a Tello: the equivalents are the emergency motor-cut and pulling the battery.

8.1 Thresholds, and where they come from

Threshold	Default	Provenance
Speed cap	0.6 stick	[D] conservative first-flight value
Forward-axis cap	0.15 stick	[D] tighter, because the forward scale is unmeasured
Battery floor	20 %	[D] — the source material names the rule, no number
Link-loss timeout	1.0 s	[D]
Stale-command timeout	0.30 s	[V] must stay under the driver's 0.35 s
Prolonged target loss	30 s	[D]
Altitude window	0.3 – 2.5 m	[D], but enforceable — height is measured
Forward stick scale	6.0 m/s	[!] UNMEASURED — deliberately high; see Section 7.5

Not one of these numbers is inherited from the source material, which names every rule and fixes no value. They are stated here as choices so that they can be argued with, rather than buried as defaults.

8.2 What an adversarial review found

The first version of the supervisor was reviewed against the driver's real behaviour by an agent instructed to find ways it could hurt the aircraft. It found several, each now fixed and pinned by a test. They are recorded here because each one looked correct on reading:

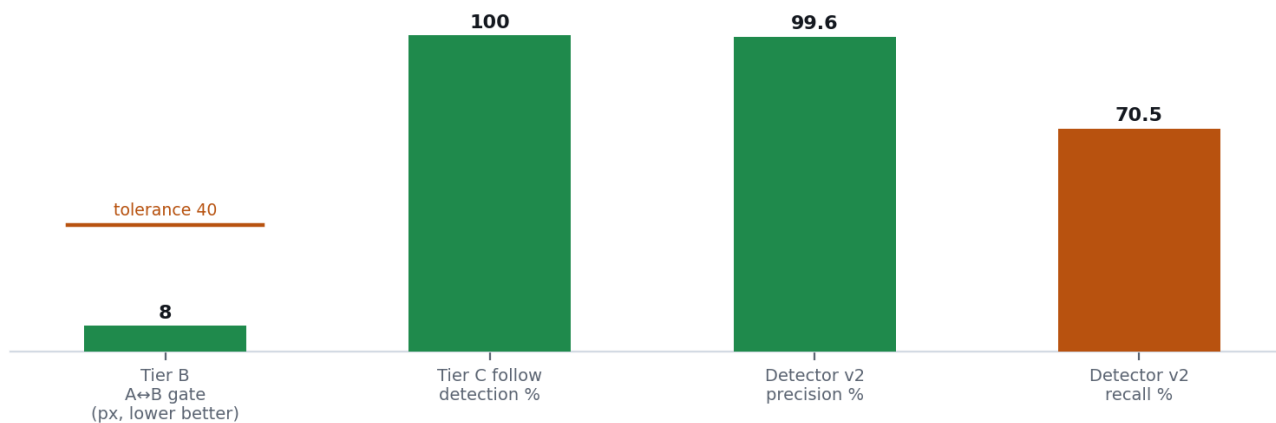
Defect	Why it was invisible	Consequence
NaN became full deflection	<code>min(0.6, nan)</code> returns 0.6 — min/max do not clamp NaN, and the "was it capped?" test compares against NaN and says no.	One bad number → +0.6 stick on every axis, reported as normal.
Target-loss rule could never fire	It keyed on detection <i>message</i> age, but the detector sends a message every frame including misses — ~33 ms, forever.	A lost target patrols until the battery dies.
Every rule failed open	A rule whose input had never arrived was skipped rather than tripped.	Arm before the driver connects → a live policy, unsupervised.
Level-triggered arming	<code>ros2 topic pub</code> without -1 republishes at 1 Hz.	Every latched fault cleared once a second, forever.
Altitude floor on the ground	Height reads ~0 before take-off.	Could not arm — inviting the operator to disable the floor.
/emergency did not disarm	The motor cut went to the driver only.	A live policy resumed the moment anyone sent <code>/takeoff</code> .

Why none of this was caught by the original tests

The integration tests had to switch every auto-disarm rule off in order to exercise the control path — they have no driver to supply telemetry. So the supervisor's fault logic was, until the review, entirely untested. A second test file now does the opposite: it drives the supervisor alone and asserts that each rule fires, and that each one fails closed.

9 Results, and what is not yet proven

The honest ledger.



The measured results across the three tiers.

Claim	Evidence
The behaviour is learnable, and RL beats the classical baseline	Tier A matrix, 20 runs, interval rule → ACCEPT
The training world matches real physics	Tier B A↔B gate on real Gazebo, 8.0 px median vs 40 px
A detector can be built entirely in simulation, and made honest	v2 precision 0.996 with drone-free frames in the exam
The full vision loop works with nothing mocked	Follow: 100 % detection over 1,500 frames
The intercept mission can genuinely complete	Verified capture at 0.48 m , frame on record
The flight stack behaves correctly without hardware	18 flight-stack tests, including every supervisor fault rule; 105 total

9.1 What is NOT proven

This is the more important half of the section, and it is the reason the system is not yet cleared to fly the mission.

Gap	Why it matters	What closes it
The detector has never seen reality. All 3,363 training images are rendered.	Rendered-to-real is the single most common failure surface for drone detectors. A precision of 0.996 on Unreal frames says nothing about a real warehouse camera.	Record real footage of the target, mix it in, retrain, re-validate.
The aircraft's real dynamics are unmeasured. The policy trained against a 0.25 s velocity lag placeholder.	Tier B <i>measured</i> 0.54–0.84 s in physics alone. A chase policy is a tight feedback loop; feeding it two to three times the delay it learned on can oscillate.	Step-response tests, then recalibrate Tier B and fine-tune if the gap demands it.
The forward stick scale is a guess.	It sets the magnitude of the only axis that decides closing energy.	The same step-response test.

The summary judgement

What exists is a **validated policy** and a **validated physics model**, plus a detector and a command path that have **not yet met reality**. That is a strong place to begin the hardware phase — and not a licence to skip it.

10 Bring-up procedure

The flight ladder, in the order it must be climbed.

Each rung produces a measurement that the next rung depends on. The safety supervisor and a human on the abort are live throughout.

#	Step	Produces	Needs
1	Measure the aircraft. Step-response tests for command lag and full-stick speed on each axis.	The two numbers every placeholder in this document is waiting for.	The drone, a tape measure, no RL
2	Recalibrate Tier B to the measured lag, re-run the A↔B gate, fine-tune the policy in Gazebo if the gap is large.	A policy trained against the real delay.	Gazebo only
3	Collect real frames of the target at known ranges; mix into the dataset; retrain the detector; re-validate on real images.	A detector that has seen reality.	Manual flight
4	Fly the P-controller first (baseline:=p_controller). Same graph, same safety chain, no learned policy.	Proof the whole chain works, plus the baseline number that makes the RL result interpretable.	Full stack, tethered
5	Fly the learned policy against a stationary target, then a slow one, then the full scenarios.	The result.	Full stack

10.1 Before every flight

- Launch with `control:=false` so the keyboard teleop node cannot fight the policy for the same command register — they write the same register and last writer wins.
- Confirm the stack comes up **DISARMED** and that `/cmd_vel` is exactly zero before arming.
- Confirm `/emergency` is wired to something you can physically reach.
- Check the detector's reference width matches the policy's (Section 7.3) — the launch file does this, but verify it.
- Run the sign test for the tier you are about to use. Three simulators plus one aircraft is four chances for a silent flip.

10.2 Commands

```
# build
colcon build --packages-select chase_msgs chase_flight tello_msg

# fly the hand-written baseline first (ladder step 4)
ros2 launch chase_flight chase_flight.launch.py \
  baseline:=p_controller weights:=<detector.pt>

# fly the learned policy (ladder step 5)
ros2 launch chase_flight chase_flight.launch.py \
  checkpoint:=<bundle.pt> weights:=<detector.pt>

# arm / abort / motor cut
ros2 topic pub -1 /chase/arm    std_msgs/Bool  'data: true'
ros2 topic pub -1 /chase/abort std_msgs/Empty '{}'
ros2 topic pub -1 /emergency   std_msgs/Empty '{}'

# watch what the supervisor is deciding
ros2 topic echo /chase/safety
```


A Reference tables

A.1 Where the code lives

Package	Role
chase_gym	THE contract: observation assembler, forward law, reward, kinematics, corruption, target motion, baselines, constants. Imported by every tier.
chase_train	The trainer: TD3/DDPG arms, checkpoint bundles, ONNX export, the run matrix and its acceptance report.
chase_eval	Held-out evaluation and the selection metric.
chase_sim_gz	Tier B: Gazebo backend, oracle detector, latency shim, sign test, A↔B replay gate.
sim/tier_c_airsim	Tier C: engine adapter, warehouse builder, dataset factory, detector trainer, vision-in-the-loop evaluation.
chase_detector	The deployed YOLO node.
chase_flight	The flight stack — the six nodes of Section 7.
chase_msgs	Interface messages: DroneDetection, TrackingState, PolicyAction, FlightMode, SafetyStatus.
tello	The driver: video, telemetry, RC output, the 0.35 s dead-man.

A.2 Topics

Topic	Type	Published by
/image_raw	sensor_msgs/Image	tello driver, 960×720 bgr8
/chase/detection	chase_msgs/DroneDetection	chase_detector, every frame
/chase/observation	chase_msgs/TrackingState	chase_state, 10 Hz
/chase/action	chase_msgs/PolicyAction	chase_policy
/chase/forward_cmd	geometry_msgs/TwistStamped	chase_depth_rule, m/s
/chase/mode	chase_msgs/FlightMode	chase_behaviour
/chase/cmd_raw	geometry_msgs/TwistStamped	chase_mixer, sticks
/cmd_vel	geometry_msgs/Twist	chase_safety ONLY , 20 Hz
/chase/action_sent	chase_msgs/PolicyAction	chase_safety → chase_state
/chase/safety	chase_msgs/SafetyStatus	chase_safety
/chase/arm, /chase/abort	std_msgs/Bool, Empty	the operator

A.3 Key constants

Constant	Value	Meaning
FX, FY	919.42 px	Focal length, from the calibration file
FRAME_W × H	960 × 720	Calibration frame
HFOV / VFOV	55.1° / 42.8°	Derived, not the ~82° usually quoted
DT	0.1 s	Control period = simulator step

Constant	Value	Meaning
K_ACTION_HISTORY	4	$\text{ceil}(0.35 \text{ s} \times 10 \text{ Hz})$
TELLO_BODY_W	0.098 m	The reference width the policy uses
TELLO_PROP_SPAN	0.180 m	The detector's default — 1.84× larger
LATENCY_RANGE_S	0.150 – 0.350 s	Measured video-link latency
DEADMAN_S	0.35 s	Driver zeroes a staler command

Every figure in this document was generated from source by `docs/make_chase_figures.py`, and the document itself by `docs/make_chase_reference.py`. Numbers marked [D] are design choices, [!] are unmeasured placeholders, and [V] are verified against code or measurement.